Title: Device Management System; Design Document, Planning, and System Requirements.

Module code: CMP5387

Team members		
Student Name	Student ID	Student Email
Seyed Atta Rahimi	23241471	Seyed.Rahimi@mail.bcu.ac.uk
Sepehr Kalantari Soltanieh	24124891	sepehr.kalantarisoltanieh@mail.bcu.ac.uk

Abstract

The following document aims to explain the Device Management System, how it works, and who the target audience is, and how the system offers features that other management systems may lack or not perform effectively. It includes all the designs used and the plans created for the system to operate efficiently and deliver the best possible results to users, as well as the required software. The design plans include but are not limited to, an Entity Relationship Diagram (ERD) in Crow's feet style, a table outlining and explaining system requirements, and a Gantt chart illustrating the project timeline and the assigned tasks for each group member. The document will also discuss the design and operation of the API being used. Additionally, it will consider the system's strengths and areas for future improvement.

Introduction

In a society where corporations are growing larger every day, the number of devices each entity may need also increases, which can pose long-term challenges in tracking them all. To address these issues, device management systems were introduced to the market, making it much easier and more straightforward for any company to keep track of all their devices, whether big or small. These systems enable seamless identification, addition, deletion, and monitoring of devices within a corporation. Examples include Ivanti Unified Endpoint Management, which allows IT teams to securely gain a clearer understanding of their endpoints by viewing all internet-connected devices within their organisation (Ivanti, n.d.). Such tools detect faulty devices and those in need of troubleshooting automatically. However, these systems are limited in that they cannot identify all problems that a typical user might encounter with a device. While automation of such tasks is generally preferred, it is undeniable that these algorithms have not yet managed to detect every issue that may arise. For instance, problems like dead pixels on a monitor cannot be easily identified by system-level algorithms, as they are physical issues that can only currently be detected by end-users. This is one simple example among many problems requiring human detection and intervention. The best solution is to enable endpoint-users to communicate directly with IT personnel and report issues, allowing them to be addressed promptly and preventing disruptions within the organisation. This document aims to discuss such a solution. A Device Management System (DMS) that allows everyone within an organisation to report device issues directly to IT staff remotely, facilitating swift servicing of those devices. This system utilises a database to record each individual in the company and all devices, including their precise locations. Such access simplifies the discovery and management of devices, especially in large organisations with multiple buildings, rooms, and thousands of devices. Each user has an account to access their dashboard and report any device issues, with reports automatically forwarded to the appropriate IT personnel. This essay will discuss and analyse the system requirements, supported by visual aids like tables to demonstrate how the system's performance is measured against each requirement. Following this, the relevant Entity Relationship Diagram (ERD) will be presented, using the Crow's feet notation, as this style is most widely adopted for such systems. The diagram will be explained in detail. Afterwards, a section will describe the design of a backend web service API (Application Programming Interface), with a subsequent discussion explaining its functionality and role within the system. Finally, a Gantt chart outlining the complete project timeline will be provided, along with an explanation of each group member's role and the rationale behind task assignments.

System Overview

It is imperative to understand what a Device Management System does, how it operates, and why it is important. A Device Management System is usually a backend service that allows for the management, monitoring, registration, and tracking of devices within an organisation. These systems typically comprise three main parts: the management interface, the communication protocol, and the agent. Administrators use the management interface to oversee devices within a network. The agent communicates with the management interface to directly control devices in an organisation or network. Finally, the communication protocol transmits management data such as configuration, status, and identification between the agent and management interfaces (Lin et al. 2016). The diagram below offers a visual overview of these three components.

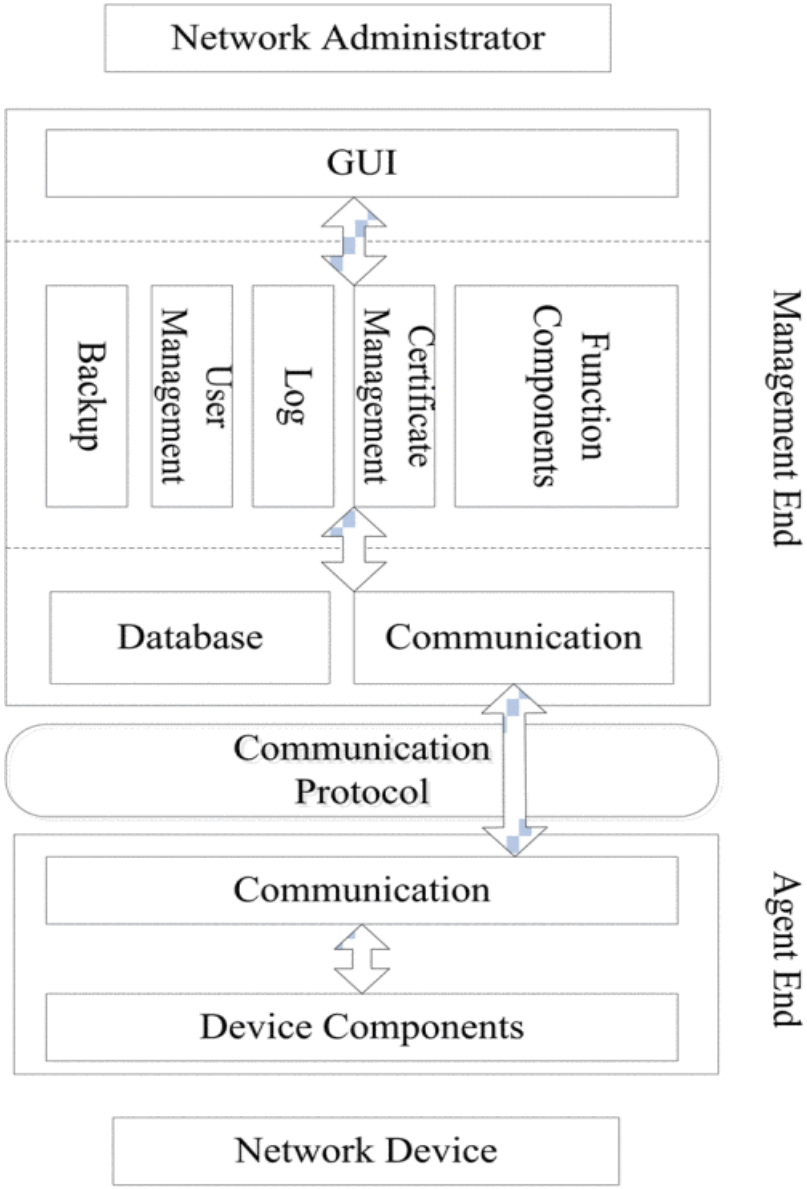

Figure 1: Structure of the network device management system (Lin et al. 2016).

These systems are interacted with by various actors such as IT technicians, device administrators, general staff, and so on. These systems should always be scalable, secure, and transparent, while providing low overhead. By using an API, the data stored in the backend will be accessible by external clients such as front-end applications (GUI).

System Requirements

Every system requires many checklists to be ticked off so it can function as required. These requirements may include: who are the users? What do they need? How will the features be implemented? etc. This Device Management System (DMS) needs interactions between users and devices. The first thing to identify is who the users are.

This system is primarily designed for any organisation with any number of staff and devices, demonstrating potential for future expansion without issues. The main users are, but not limited to, staff and IT technicians, as any user can report a fault with a device, and the nearest IT technician will be assigned to the case, ensuring timely resolution. The system enables direct interactions between users and IT staff through various processes such as device/request assignment, fault reporting, and maintenance monitoring.

User / Stakeholder	Description	Problem(s)	Potential Benefit
Technician / Maintenance Staff	Personnel responsible for diagnosing, repairing, and maintaining hardware and software systems.	P1: Delays in receiving service requests and unclear issue descriptions cause repair inefficiencies. P2: Lack of a structured tracking system for ongoing and completed repairs.	Instant notification of new service requests, access to detailed device information, and the ability to update repair status in real time through the system.
Staff / End Users	General employees or academic staff who use computers and devices daily or any type of user that may be given access to the system.	P3: No easy way to report device issues or request updates and repairs. P4: Lack of visibility over progress of their service requests.	A simple web interface to submit requests, view status updates, and communicate with the IT department.

Table 1: Stakeholder Table

Table 1 displays all users and stakeholders who may use this system and the problems it can solve to make their tasks more efficient and their lives easier.

Next, a set of functionalities needs to be identified and implemented; however, not all functions have the same priority. Some functions take implementation priority over others to ensure core functionality and an optimal user experience.

ID	Feature	Description	Problem(s) Solved	Users / Stakeholders Involved	Importance
F1	User Authentication and Role Management	Ensures secure login and assigns different access levels (Technician, Staff/End users) to protect data and maintain role-based control.	General (system access control)	All Users	Must have
F2	Service Request Submission	Enables staff/end users to report issues and request updates or repairs through a simple	P2, P3	Staff/End users	Must have

		web form without attachments.			
F3	Request Tracking Dashboard	Displays all repair or maintenance requests with real-time status indicators (Pending, In Progress, Completed).	P3, P4	IT Technicians, Staff/End users	Must have
F4	Technician Task Management	Allows technicians to view assigned requests, update progress, add notes, and mark tasks as completed.	P1, P2	IT Technicians	Must have
ID	Feature	Description	Problem(s) Solved	Users / Stakeholders Involved	Importance

Table 2: Functional requirements table

The functional requirements table shown above is directly made to address the issues faced by users. User Authentication and Role Management (F1) are implemented to establish a secure foundation for system access and role-based data access.

Service Request Submission (F2) and Request Tracking Dashboards (F3) are implemented to enhance communication between users and technicians, which reduces delays in repairs and fault reports.

The Technician Task Management feature (F4) enables structured workflows by helping maintenance personnel manage workloads efficiently, while Audit Logging and Data Backup (F5) improve accountability, traceability, and system resilience. Overall, these features allow the system not only to support device fault reporting but also to provide a scalable, future-proof, and secure infrastructure for organisational device management.

Entity Relationship Modelling

This section includes a complete entity relationship diagram (ERD) in the style of crow's feet, which is currently the most widely used modelling method to represent databases. This model is necessary to ensure proper documentation and planning for the database that will enable the Device Management System (DMS) to function by storing and retrieving information.

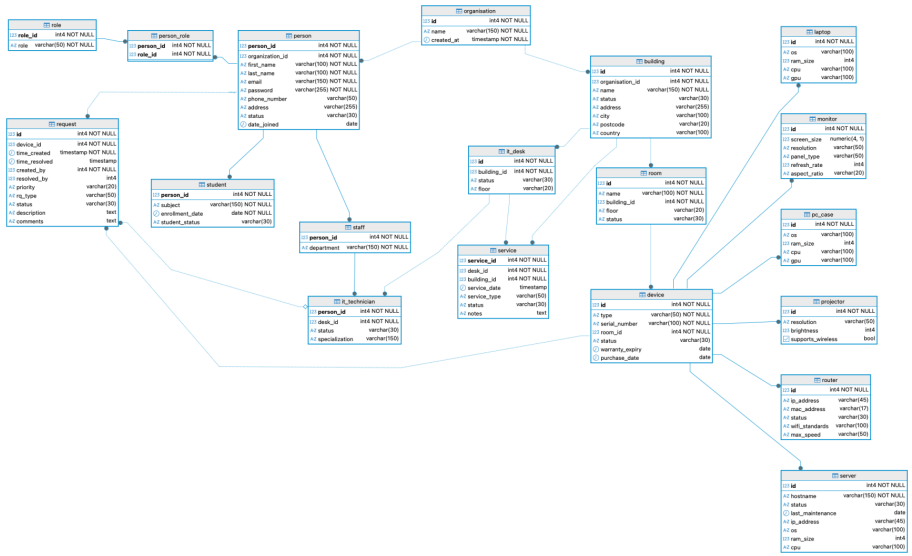

Figure 2: Entity Relationship Diagram (ERD) for the Device Management System. (Appendix A)

Figure 2 shows the tables and their relationships using relationship lines and cardinality indicators. The parent table is the organisation, which connects to the person table that can have subtypes such as a student or staff. It also connects to the building table, which connects to the rooms table. The rooms table then connects to the device table, which can have many subtypes like laptops, projectors, monitors, and so on. The staff table is managed by staff, as managers are also considered staff, and an extension of staff are the IT-Technicians, all assigned to an IT-desk. These IT-desks are within a specific building, but each IT-desk can service one or more buildings simultaneously. There is also a request table used to

store and receive requests from any person, which are directed to IT-technicians for resolution. The request table also has a direct relationship with the device table, as a request must be related to a specific device.

Backend API Design

This section provides a clear overview of all web service endpoints within the Device Management System. Each endpoint represents a unique resource or operation accessible via the REST API, collectively defining the full interaction surface between client applications and the backend system. To ensure clarity and consistency, all URIs follow a versioned namespace (/api/v1/) and adhere to REST principles in their use of HTTP methods such as GET, POST, and PATCH. The following inventory summarises all available endpoints, including their purpose, expected behaviour, and communication flows.

-Resource URI Summary Table

Endpoint (URI)	HTTP Method	Description / Purpose
`/api/v1/devices`	GET	Retrieves a list of all devices stored in the system, including device type, serial number, status, room, and building information.
`/api/v1/devices/{device_id}`	GET	Retrieves detailed information about a single device, including its location and associated building.

`/api/v1/devices`	POST	Creates a new device record with attributes such as type, serial number, status, and room assignment.
`/api/v1/requests`	POST	Creates a new support request for a device, including priority, type, description, and creator information.
`/api/v1/requests/{req_id}/resolve`	PATCH	Marks an existing request as resolved by assigning a technician, adding resolution comments, and saving a completion timestamp.
`/api/v1/login`	POST	Authenticates a user by validating email and password and returns associated roles for authorisation.
`/api/v1/requests`	GET	Retrieves a full list of all support requests, including device details, creator information, location, and resolver information (when applicable).

`/api/v1/requests/createdby/{person_id}`	GET	Retrieves all support requests created by a specific person, including related device and location details.
`/api/v1/requests/resolved-by/{tech_id}`	GET	Lists all support requests resolved by a specific IT technician.
`/api/v1/requests/{req_id}`	GET	Retrieves full details of a single support request, including device, creator, location, and resolver information.
`/api/v1/technicians`	GET	Returns a list of all registered IT technicians, including names and contact information.

Table 3: URI Summary Table

-Protocol Specification

The web service uses a REST-based communication model, where all interactions occur via HTTP requests and JSON-encoded payloads. Each endpoint is accessed through a clearly defined URI structure within the /api/v1/ namespace, indicating both the resource being accessed and the API version. All clients interacting with the service must include the Content-Type: application/json header for POST and PATCH operations, while GET requests utilise URI parameters and do not require a request body. Likewise, all endpoints return data in JSON format, ensuring consistent serialisation across the system. Regarding input validation, each endpoint defines a strict structure for what constitutes a valid request. POST and PATCH operations require specific fields within the JSON body—for instance, the POST /api/v1/requests endpoint must include device_id, created_by, priority, rq_type, and description. Missing or invalid fields result in a 400 Bad Request response with a structured JSON error message. Endpoints that rely on existing database records (such as device IDs, person IDs, and technician IDs) perform additional referential checks before processing the request. These validations prevent the insertion of incomplete or inconsistent data and ensure that client errors are communicated promptly and clearly. The API also standardises its error-reporting mechanism through a dedicated json_error() helper, which produces uniform error objects such as { "error": "Device not found" } along with an appropriate status code. This consistency applies across all endpoints, including authentication failures (401 Unauthorised), resource-not-found scenarios (404 Not Found), and unexpected database issues (500 Internal Server Error). By returning well-defined status codes and structured JSON objects, the service effectively communicates the nature and severity of errors in a predictable way that client applications can reliably interpret. All database operations are performed using parameterised SQL queries through the psycopg2 driver. This guarantees that user-supplied values are never directly inserted into SQL statements, reducing the risk of SQL injection attacks and enhancing overall security. On successful operations, endpoints return either a 200 OK status (for retrieval or update) or 201 Created for resource creation, often with identifiers or confirmation messages. These protocol rules together create a clear, predictable, and secure communication interface between clients and the Device Management System backend.

Endpoint	HTTP Method	Description	Example Input (Request)	Example Output (Response)	Status Code	Happy /Sad

/api/v1/devices	GET	Retrieves a JSON array containing all devices stored in the database, including device type, serial number, status, room, and building.	Headers:Accept: application/json
			Body: N/A
				[
				{
				"id": 1,
				"type": "laptop",
				"serial_number": "LAP123",
				"status": "in_use",
				"room": {
				"id": 1,
				"name": "IT-101"
				},
				"building": {
				"id": 1,
				"name": "Curzon Building"
				}
				},
				{
				"id": 2,
				"type": "router",
				"serial_number": "RT5567",
				"status": "active",
				"room": {
				"id": 1,
				"name": "IT-101"
				},
				"building": {
				"id": 1,
				"name": "Curzon Building"
				}
				}
				]
					200	Happy

/api/v1/devices/{id}	GET	Retrieves detailed information for a single device, identified by device ID.	Headers:Accept: application/json	{ "id": 1, "type": "laptop", "serial_number": "LAP123",	200	Happy

			URL Example:/api/v1/devices/1	"status": "in_use",
				"room": {
				"id": 1,
				"name": "IT-101"
				},
				"building": {
				"id": 1,
				"name": "Curzon Building"
				}
				}		

/api/v1/devices/9999	GET	Attempt to retrieve a device that does not exist	Headers: Accept: application/json URL: /api/v1/devices/9999	{ "error": "Device not found", "message": "No device exists with ID 9999" }	404	Sad
/api/v1/devices/abc	GET	Invalid device ID format (non-numeric)	Headers: Accept: application/json URL: /api/v1/devices/abc	{ "error": "Invalid device ID", "message": "Device ID must be a numeric value" }	400	Sad
/api/v1/devices	POST	Creates a new device record in the database. Requires device type, serial number, and room ID. Additional optional fields include status, warranty expiry, and purchase date.	Headers:Content-Type: application/jsonBody:json\n{\n "type": "laptop",\n "serial_number": "LAP999",\n "room_id": 1,\n "status": "in_use",\n "warranty_expiry": "2026-12-01",\n "purchase_date": "2023-09-15"\n}\n	json\n{\n "message": "Device created",\n "device_id": 7\n}\n	201	Happy
/api/v1/devices	POST	Missing required fields: type, serial_number, or room_id.	Headers:Content-Type: application/jsonBody:json\n{\n "type": "laptop"\n}\n	json\n{\n "error": "Missing required field(s): serial_number, room_id"\n}\n	400	Sad
/api/v1/devices	POST	Fails if an invalid room_id is provided.	Headers:Content-Type: application/jsonBody:json\n{\n "type":	json\n{\n "error": "Room not found"\n}\n	404	Sad

			"router",\n "serial_number": "R001",\n "room_id": 9999\n}\n			
/api/v1/devices	POST	Fails when the serial number already exists in the database.	Headers:Content-Type: application/jsonBody:json\n{\n "type": "monitor",\n "serial_number": "LAP123",\n "room_id": 1\n}\n	json\n{\n "error": "Device with this serial_number already exists"\n}\n	400	Sad
/api/v1/requests	POST	Creates a new support request for a specific device. The request must include device ID, creator ID, priority, request type, and description. If valid, the request is stored with status "open".	Headers:Content-Type: application/jsonBody:json\n{\n "device_id": 1,\n "created_by": 3,\n "priority": "high",\n "rq_type": "hardware",\n "description": "Laptop screen is flickering."\n}\n	json\n{\n "message": "Request created",\n "request_id": 12,\n "status": "open"\n}\n	201	Happy
/api/v1/requests	POST	Fails if one or more required fields are missing (device_id, created_by, priority, rq_type, description).	Headers:Content-Type: application/jsonBody:json\n{\n "device_id": 1,\n "priority": "high"\n}\n	json\n{\n "error": "Missing required field(s): created_by, rq_type, description"\n}\n	400	Sad
/api/v1/requests	POST	Fails if device_id does not exist in the device table.	Headers:Content-Type: application/jsonBody:json\n{\n "device_id": 9999,\n "created_by": 3,\n "priority": "medium",\n "rq_type": "software",\n "descrip	json\n{\n "error": "Device not found"\n}\n	404	Sad

			tion": "Unknown device test."\n}\n			
/api/v1/requests	POST	Fails when the provided created_by ID does not exist in the person table.	Headers:Content-Type: application/json Body:json\n{\n "device_id": 1,\n "created_by": 9999,\n "priority": "high",\n "rq_type": "hardware",\n "description": "Testing invalid person."\n}\n	json\n{\n "error": "Creator person not found"\n}\n	404	Sad
/api/v1/requests	POST	Returns a general error message for unexpected database failures.	Headers:Content-Type: application/json Body:json\n{\n "device_id": 1,\n "created_by": 1,\n "priority": "medium",\n "rq_type": "hardware",\n "description": "Test"\n}\n	json\n{\n "error": "Database error: <error_message_here>"\n}\n	500	Sad

/api/v1/requests/{id}/resolve	PATCH	Marks an existing support request as resolved by a valid IT technician. Updates the status to "resolved", stores the technician who resolved it, the resolution time, and optional comments.	Headers:Content-Type: application/json URL Example:/api/v1/requests/12/resolve Body:json
			{
			 "resolved_by": 2,
			 "comments": "Reinstalled drivers and tested the device."
			}
			
				json
				{
				 "message": "Request resolved",
				 "request_id": 12,
				 "status": "resolved"
				}
				
					200	Happy

/api/v1/requests/{id}/resolve	PATCH	Fails when the required resolved_by field is missing from the request body.	Headers:Content-Type: application/json URL Example:/api/v1/requests/12/resolve Body:json
			{
			 "comments": "Done."
			}
			
				json
				{
				 "error": "Missing required field: resolved_by"
				}
				
					400	Sad

/api/v1/requests/{id}/resolve	PATCH	Fails if the specified request ID does not exist in the request table.	Headers:Content-Type: application/json URL Example:/api/v1/requests/9999/resolve Body:json
			{
			 "resolved_by": 2,
			 "comments": "Tried to resolve."
			}
			
				json
				{
				 "error": "Request not found"
				}
				
					404	Sad

/api/v1/requests/{id}/resolve	PATCH	Fails if the resolved_by person ID is not present in the it_technician table.	Headers:Content-Type: application/json URL Example:/api/v1/requests/12/resolve Body:json
			{
			 "resolved_by": 999,
			 "comments": "Tried to resolve."
			}
			
				json
				{
				 "error": "Resolved_by must be a valid IT technician"
				}
				
					400	Sad

/api/v1/login	POST	Authenticates a user based on email and password. On success, returns a success message, basic user details, and a list of their roles.	Headers:Content-Type: application/json Body:json
			{
			 "email": "alex.smith@bcu.ac.uk",
			 "password": "pass123"
			}
			
				{ "message": "Login successful", "person_id": 1, "first_name": "Alex", "last_name": "Smith", "roles": ["student", "staff"] }	200	Happy

/api/v1/login	POST	Fails if one or both required fields (email, password) are missing from the request body.	Headers:Content-Type: application/json Body:json
			{
			 "email": "alex.smith@bcu.ac.uk"
			}
			
				json
				{
				 "error": "Missing required field(s): password"
				}
				
					400	Sad

/api/v1/login	POST	Fails if the email does not exist or the password does not match the stored value. For security, the same generic error message is returned in both cases.	Headers:Content-Type: application/json Body:json
			{
			 "email": "alex.smith@bcu.ac.uk",
			 "password": "wrongpass"
			}
			
				json
				{
				 "error": "Invalid email or password"
				}
				
					401	Sad

/api/v1/requests	GET	Retrieves a JSON array containing all support	Headers:Accept: application/json	[{	200	Happy

		requests stored in the system, including device, location, creator, and (optionally) resolver details.	Body: N/A	"request_id": 1,
				"priority": "high",
				"type": "hardware",
				"status": "open",
				"description": "Laptop not turning on.",
				"created_at": "2024-11-30T10:15:00",
				"time_resolved": null,
				
				"device": {
				"id": 1,
				"type": "laptop",
				"serial_number": "LAP123"
				},
				
				"location": {
				"building": {
				"id": 1,
				"name": "Curzon Building"
				},
				"room": {
				"id": 1,
				"name": "IT-101"
				}
				},
				
				"created_by": {
				"id": 3,
				"first_name": "Alex",
				"last_name": "Smith"
				},
				
				"resolved_by": null
				}
				]		

/api/v1/requests/created-by/{person_id}	GET	Retrieves all support requests created by a	Headers:Accept: application/json\URL	{ "created_by": {	200	Happy

		specific person, identified by their person_id. Includes device, room, building, and status details.	Example:/api/v1/requests/created-by/3
			Body: N/A
				"id": 3,
				"first_name": "Alex",
				"last_name": "Smith"
				},
				"requests": [
				{
				"request_id": 5,
				"device": {
				"id": 1,
				"serial_number": "LAP123",
				"type": "laptop"
				},
				"location": {
				"room": {
				"id": 1,
				"name": "IT-101"
				},
				"building": {
				"id": 1,
				"name": "Curzon Building"
				}
				},
				"priority": "high",
				"type": "hardware",
				"status": "resolved",
				"description": "Laptop not turning on.",
				"created_at": "2024-11-29T09:30:00",
				"resolved_at": "2024-11-29T11:10:00"
				},
				{
				"request_id": 8,
				"device": {
				"id": 2,
				"serial_number": "RT5567",
				"type": "router"
				},		

				"location": {
				"room": {
				"id": 1,
				"name": "IT-101"
				},
				"building": {
				"id": 1,
				"name": "Curzon Building"
				}
				},
				"priority": "medium",
				"type": "network",
				"status": "open",
				"description": "Intermittent Wi-Fi connection.",
				"created_at": "2024-11-30T14:05:00",
				"resolved_at": null
				}
				]
				}		

/api/v1/requests/created-by/{person_id}	GET	Returns an error if the specified person_id does not exist in the person table.	Headers:Accept: application/json URL Example:/api/v1/requests/created-by/9999 Body: N/A	json\n{\n \"error\": \"Person not found\"\n}\n	404	Sad

/api/v1/requests/resolved-by/{tech_id}	GET	Returns all support requests that were resolved by the specified IT technician. Includes device, room, building, timestamps, and full technician info.	Headers:Accept: application/jsonURL Example:/api/v1/requests/resolved-by/2
			Body: N/A
				{
				"technician": {
				"id": 2,
				"first_name": "John",
				"last_name": "Doe"
				},
				"resolved_requests": [
				{
				"request_id": 10,
					200	Happy

				"device": {
				"id": 3,
				"serial_number": "SRV890",
				"type": "server"
				},
				
				"location": {
				"room": {
				"id": 3,
				"name": "ENG-305"
				},
				"building": {
				"id": 2,
				"name": "Millennium Point"
				}
				},
				
				"priority": "high",
				"type": "hardware",
				"status": "resolved",
				"description": "Server overheating.",
				"created_at": "2025-01-10T09:10:00",
				"resolved_at": "2025-01-10T09:35:00"
				}
				]
				}		

/api/v1/requests/resolved-by/{tech_id}	GET	Fails when the supplied technician ID is not found in it_technician.	URL Example:/api/v1/requests/resolved-by/9999 Body: N/A	json\n{\n \"error\": \"Technician not found\"\n}\n	404	Sad
/api/v1/requests/resolved-by/{tech_id}	GET	Technician exists, but there are no resolved requests in the system for them.	URL Example:/api/v1/requests/resolved-by/5	json\n{\n \"technician\": {\n \"id\": 5,\n \"first_name\": \"Fatima\",\n \"last_name\": \"Khan\"\n },\n \"resolved_requests\": []\n}\n	200	Happy (valid request, empty results)

/api/v1/requests/{id}	GET	Retrieves full details for a single support request, including device, location, creator, and (if available) resolver information.	Headers:Accept: application/json
			
			URL Example:/api/v1/requests/10
			Body: N/A
				{
				"request_id": 10,
				"priority": "high",
				"type": "hardware",
				"status": "resolved",
				"description": "Server overheating.",
				"comments": "Cleaned fans and improved airflow.",
				"created_at": "2025-01-10T09:10:00",
				"resolved_at": "2025-01-10T09:35:00",
				
				"device": {
				"id": 3,
				"serial_number": "SRV890",
				"type": "server"
				},
				
				"location": {
				"room": {
				"id": 3,
				"name": "ENG-305"
				},
				"building": {
				"id": 2,
				"name": "Millennium Point"
				}
				},
				
				"created_by": {
				"id": 5,
				"first_name": "Alex",
				"last_name": "Smith"
				},
				
				"resolved_by": {
				"id": 2,
					200	Happy

				"first_name": "John", "last_name": "Doe" } }		
/api/v1/requests/{id}	GET	Returns an error if the requested support request ID does not exist in the database.	Headers:Accept: application/json URL Example:/api/v1/requests/9999 Body: N/A	json\n{\n \"error\": \"Request not found\"\n}\n	404	Sad

/api/v1/technicians	GET	Retrieves a JSON array of all IT technicians in the system, including their basic personal information (name, email, phone).	Headers:Accept: application/json
			
			Body: N/A
				[
				{
				"person_id": 2,
				"first_name": "John",
				"last_name": "Doe",
				"email": "john.doe@bcu.ac.uk",
				"phone": "07123 456789"
				},
				{
				"person_id": 4,
				"first_name": "Fatima",
				"last_name": "Khan",
				"email": "fatima.khan@bcu.ac.uk",
				"phone": "07111 222333"
				}
				]
					200	Happy

/api/v1/technicians	GET	If there are no IT technicians stored in the database, the endpoint returns an empty JSON array.	Headers:Accept: application/json Body: N/A	json\n[]\n	200	Happy (valid but empty result)

Table 4: Endpoint Summary Table.

-Main Discussion of Prototype Code

This section outlines the design of the web service, including a comprehensive list of resource URIs, the communication protocol utilised by each endpoint, and a discussion of the prototype implementations that shaped the final design. To refine our web service design, we developed several prototype endpoints using Flask and PostgreSQL, and we iteratively improved them based on early testing. All endpoints follow a consistent pattern: they accept and return JSON payloads, utilise HTTP status codes to indicate success or failure, and access the database through parameterised SQL queries using psycopg2. This small-scale experimentation helped us validate both the URI design and the practicality of the implementation. For example, Code Snippet 1 (see Appendix C) demonstrates the implementation of the POST /api/v1/requests endpoint, which creates a new support request. In this function, we first check that the request has the Content-Type: application/json header and then parse the body using request.get_json(). We explicitly validate that all required fields (device_id, created_by, priority, rq_type, description) are present before interacting with the database. This prevents partially defined records from being inserted. We also perform foreign-key-style checks in the application code: the endpoint verifies that the referenced device exists in the device table and that the creator exists in the person table. If either lookup fails, the API returns a clear JSON error message (for example, { "error": "Device not found" }) with an appropriate HTTP status code (404 Not Found). Only after these checks succeed do we insert the new request with a default status of "open" and return 201 Created along with the newly generated request_id. We also implemented a POST /api/v1/login prototype (Code Snippet 2), which demonstrates how the API will eventually support authentication and role-based access control. The endpoint retrieves a user by email from the person table and compares the provided password with the stored value. Although our prototype uses plaintext passwords for simplicity, we designed the function to easily be extended to use hashed passwords in a production environment. Notably, the endpoint does not specify whether the email or password was incorrect; it always returns a generic "Invalid email or password" message and a 401 Unauthorised status code. This is a deliberate design decision to prevent leaking information about existing user accounts. After successful authentication, the endpoint queries the role and person_role tables to return the user’s roles (for example, "student" and "staff"), which will later be utilised to restrict access to endpoints. Overall, building these prototypes enabled us to evaluate how each endpoint behaves and to refine our overall approach to validation, error handling, database queries, and response formatting. During testing, we identified ways to improve the API, such as returning more detailed data structures (e.g., including room and building information in device and request queries) and standardising error reporting with the json_error() utility. These improvements ensured that the final API design is more reliable, predictable, and simpler to integrate with client applications.

Project Timeline and Workload Planning

To ensure the Device Management System could be implemented promptly, a plan was devised to distribute workloads among team members.

The timeline began in week 7 of the semester, at the beginning of November 2025.

After deciding on the final project idea, the first step was to identify the users, the problems they faced, and how they could be solved.

These were identified within week 7 itself, and also the needed features were decided the same week.

From there, the Entity-Relationship diagram was completed in a logical format.

The physical ERD took some weeks to complete, but was divided into subtasks for each team member to reduce the workload.

The table below shows the tasks planned for completion.

Task ID	**Task Name**	**Description**
T1	Requirements Analysis	Identify stakeholders, user needs, and system features
T2	Database Design	Create ERD and define data types
T3	API Design	Define endpoints, and methods
T4	Documentation	Documenting the designs and submitting the D2 assessment.
T5	System Front-end implementation	Designing the front-end of the system and linking it to the API
T6	Testing Plan Design	Define testing strategy and success criteria
T7	Designing the presentation and video pitch	Record a full demonstration of the system with all the group members and submit it for the D3 assessment.

Table 5: Work Breakdown Structure Table (WBS)

Table 5 lists each task to be completed and scheduled for the group. Below is a table showing which task each team member is responsible for.

Task ID	Task Name	Responsible
T1	Requirements Analysis	Seyed Atta Rahimi and Sepehr Kalantari Soltanieh
T2	Database Design	Seyed Atta Rahimi and Sepehr Kalantari Soltanieh
T3	API Design	Sepehr Kalantari Soltanieh and consultation from Seyed Atta Rahimi
T4	Documentation	Seyed Atta Rahimi and Sepehr Kalantari Soltanieh
T5	System Front-end Implementation	Seyed Atta Rahimi and consultation from Sepehr Kalantari Soltanieh
T6	Testing Plan Design	Seyed Atta Rahimi and Sepehr Kalantari Soltanieh
T7	Presentation and Video Pitch	Seyed Atta Rahimi and Sepehr Kalantari Soltanieh

Table 6: Responsibility Assignment Table.

The table above shows who is assigned which tasks. Most of the functions are being worked on collaboratively, with T3 and T5 focused on by one team member and frequent reviews and suggestions from the other.

-Gantt Chart

Project Start: Mon, 10/11/2025

Display Week: 3

TASK	ASSIGNED TO	PROGRESS	START	END
Conceptual Phase				
Requirement Analysis	Atta & Sepehr	100%	10/11/25	11/11/25
ERD Design	Atta & Sepehr	100%	11/11/25	9/12/25
Database Implementation				
Database Design	Atta & Sepehr	100%	1/12/25	5/12/25
API Design	Sepehr with Atta's consultation	100%	5/12/25	10/12/25
Documentation D2	Atta & Sepehr	100%	4/12/25	10/12/25
System Implementation				
System Front-end Implementation	Atta with Sepehr's consultation	0%	12/12/25	10/1/26
Testing Plan Design	Atta & Sepehr	0%	12/12/25	13/1/26
Finishing Phase				
Presentation and Video Pitch D3	Atta & Sepehr	0&	14/1/26	16/1/26

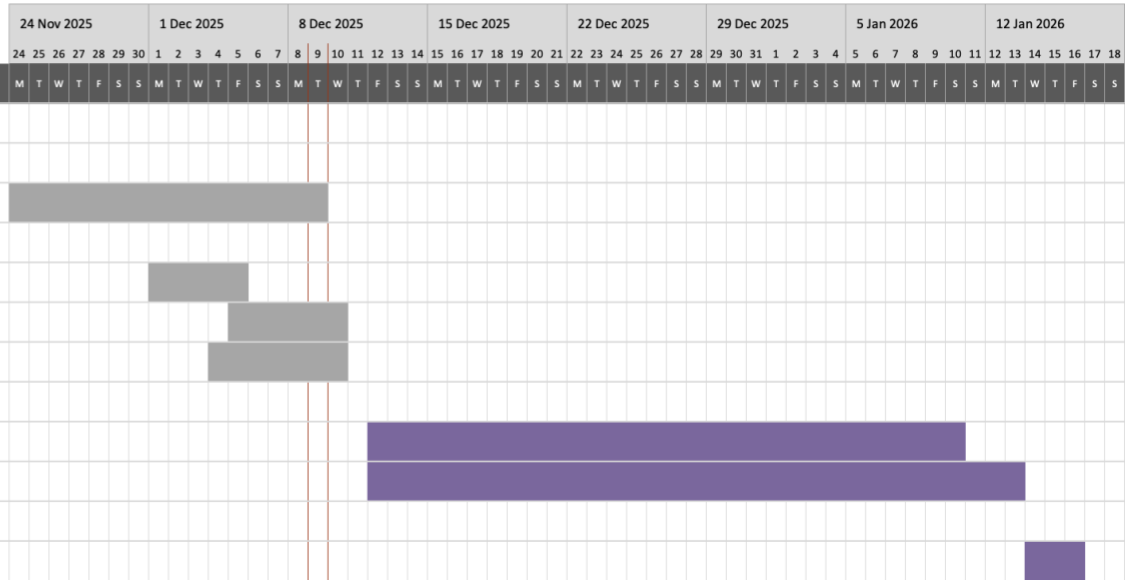

Figure 3: Project timeline Gantt chart

The above Gantt chart outlines each task's timeline and expected completion dates. This provides a basic understanding of the project's schedule. These tasks, however, depend on each other to enable other tasks to progress. In the initial phase, a requirements analysis is necessary before the database design can proceed. The API design is also heavily dependent on the completion of the database design, as the data must be precise for the API methods to be accurately developed. Documentation depends on all the preceding tasks to be comprehensive. The final three tasks depend on all earlier tasks being completed so that the team can proceed. Completing tasks such as requirement analysis and database design first ensures architectural consistency across the system, reducing the risks of future rework that could destabilise it.

References

- Ivanti. n.d. “Unified Endpoint Management Solutions.” Ivanti. Accessed November 22, 2025. https://www.ivanti.com/en-gb/unified-endpoint-management.
- Lin, Jianan, Qiaoduo Zhang, Lingcui Zhang, and Fenghua Li. 2016. “Design and Implementation of an Extensible Network Device Management System.” *International Conference on Complex, Intelligent, and Software Intensive Systems (CISIS)* 10:456-461. 10.1109/CISIS.2016.61.

Appendices

Appendix A

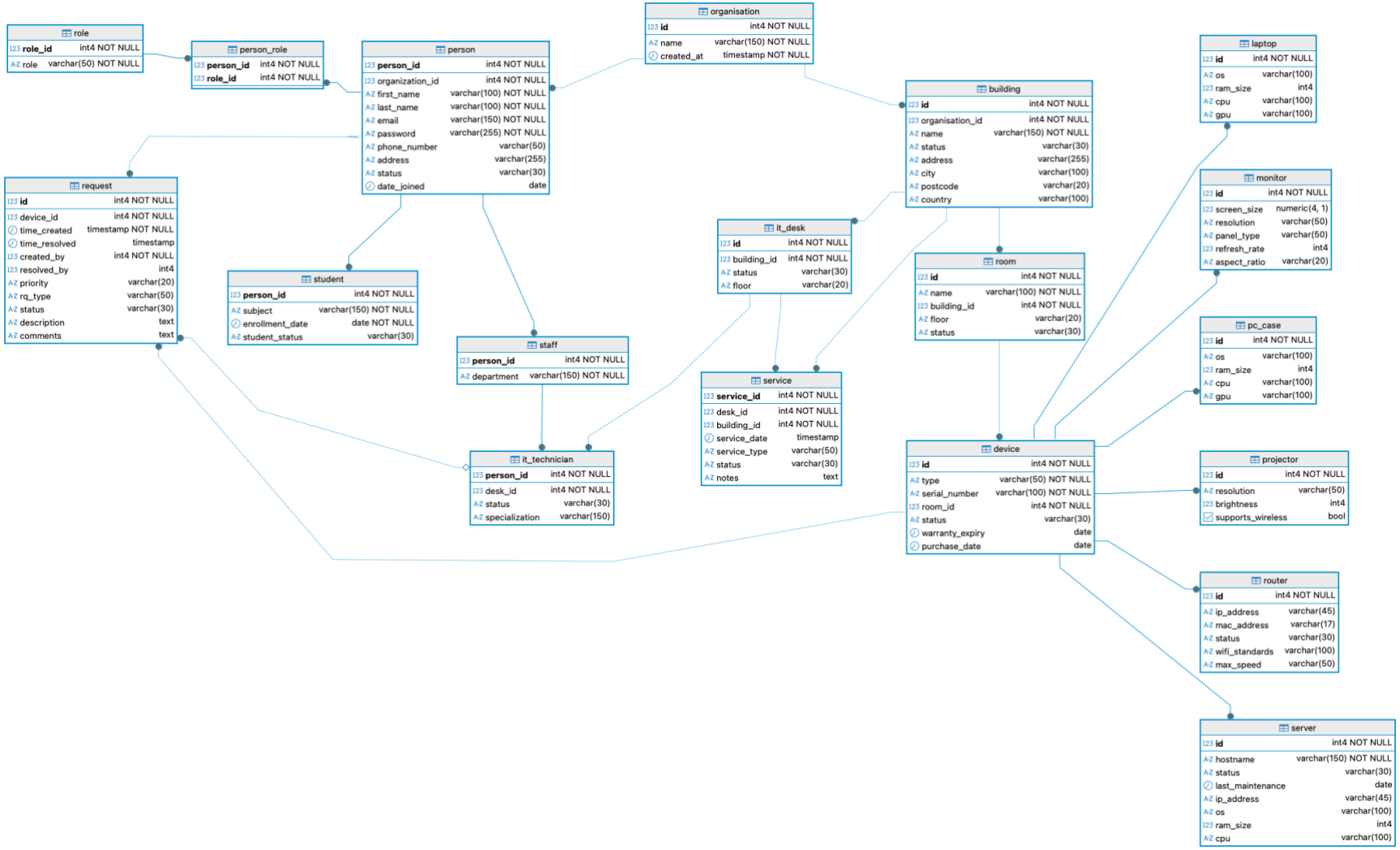

Figure 1: Appendix A ERD diagram with datatypes (DBeaver screenshot).

Appendix B

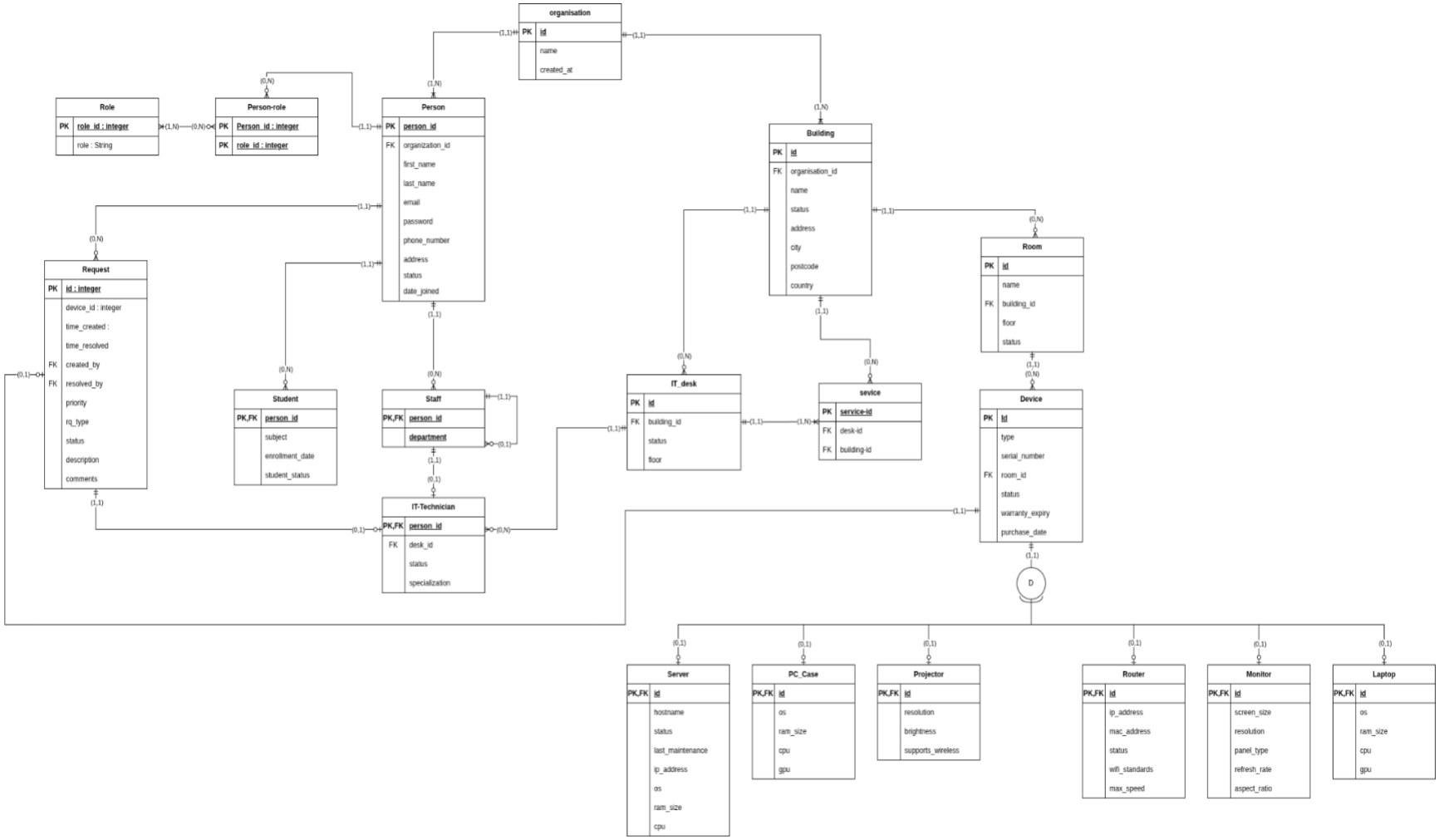

Figure 2: Appendix B ERD diagram with cardinality notation. (draw.io)

Appendix C — Initial Prototyping and Source Code Snippets

Code Snippet 1 — Early Prototype for POST /api/v1/requests (unsafe version)

```
# Code Snippet 1 — Early Prototype for POST /api/v1/requests  (unsafe version)

@app.route("/api/v1/requests", methods=["POST"])
def create_request_prototype():
    data = request.get_json()

    query = f"""
        INSERT INTO request (device_id, created_by, priority, rq_type, status, description)
        VALUES ({data['device_id']}, {data['created_by']},
                '{data['priority']}', '{data['rq_type']}',
                'open', '{data['description']}')
        RETURNING id;
    """

    conn = psycopg2.connect(...)
    cur = conn.cursor()
    cur.execute(query)
    new_id = cur.fetchone()[0]
    conn.commit()

    return {"message": "Request created", "request_id": new_id}, 201
```

***Code snippet 1**: Code Snippet 1 — Early Prototype for POST requests*

Code Snippet 2 — Early Prototype for POST /api/v1/login (unsafe version)

```
# Code Snippet 2 — Early Prototype for POST /api/v1/login (unsafe version)

@app.route("/api/v1/login", methods=["POST"])
def login_prototype():
    data = request.get_json()

    email = data["email"]
    password = data["password"]

    query = f"""
        SELECT person_id, first_name, last_name, email, password
        FROM person
        WHERE email = '{email}';
    """

    conn = psycopg2.connect(...)
    cur = conn.cursor()
    cur.execute(query)
    user = cur.fetchone()

    if not user:
        return {"error": "Invalid login"}, 401

    if password != user[4]:
        return {"error": "Invalid login"}, 401
```

```
q2 = f"""
    SELECT r.role
    FROM role r
    JOIN person_role pr ON pr.role_id = r.role_id
    WHERE pr.person_id = {user[0]};
"""
cur.execute(q2)
roles_rows = cur.fetchall()
roles = [r[0] for r in roles_rows]

return {
    "message": "Login OK",
    "person_id": user[0],
    "first_name": user[1],
    "last_name": user[2],
    "roles": roles
}, 200
```

***Code snippet 2**: Early Prototype for POST login*